

UVa 331 - Mapping the Swaps

1. Explicación del Problema

El problema nos pide calcular cuántas secuencias **mínimas** de intercambios adyacentes (es decir, el mecanismo básico de ordenamiento por burbuja o *Bubble Sort*) existen para transformar un arreglo de enteros dado en un arreglo completamente ordenado de forma ascendente.

Un mapa de intercambios (*swap map*) se representa listando secuencialmente el índice del primer elemento de cada par adyacente que se intercambia. Sin embargo, dado que realizar un intercambio redundante e inmediatamente deshacerlo no altera el orden final de los elementos, el número total de mapas de intercambios posibles para cualquier arreglo desordenado es infinito. Por ello, el problema restringe la búsqueda exclusivamente a aquellos mapas que poseen el **tamaño mínimo absoluto** de operaciones necesarias para ordenar el conjunto.

Consideraciones clave:

- Si el arreglo original ya se encuentra ordenado de manera ascendente, no se requiere realizar ningún intercambio. En este caso especial, la longitud mínima de intercambios es cero, y dado que no hay operaciones válidas que conformen un mapa de tamaño mínimo, la respuesta correcta es **0**.
- Las restricciones del problema garantizan que el tamaño del arreglo (**n**) nunca será mayor a **5** (**$n \leq 5$**). Esta dimensión tan pequeña nos permite implementar una estrategia de búsqueda exhaustiva utilizando **Backtracking** sin riesgo de exceder los límites de tiempo.

2. Código Fuente en C++

```
#include <iostream>
using namespace std;

int arr[5];
int n;
int mapas_minimos;
int min_pasos;

// Función manual para verificar si el arreglo está completamente ordenado
bool estaOrdenado() {
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1]) {
            return false;
        }
    }
    return true;
}

// Backtracking para explorar las secuencias de intercambios
void buscar(int pasos) {
    // Si ya superamos el mínimo de pasos encontrado previamente, podamos la rama
    if (pasos > min_pasos) {
        return;
    }
}
```

```

    if (estaOrdenado()) {
        if (pasos < min_pasos) {
            min_pasos = pasos;
            mapas_minimos = 1; // Encontramos una nueva longitud mínima global
        } else if (pasos == min_pasos) {
            mapas_minimos++; // Es otra forma válida con la misma longitud mínima
        }
        return;
    }

    // Intentar simular un intercambio de Bubble Sort en cada posición válida
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1]) {
            // Intercambio (Swap manual)
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;

            // Llamada recursiva con el siguiente paso
            buscar(pasos + 1);

            // Deshacer el intercambio (Backtrack)
            temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
        }
    }
}

int main() {
    int caso_num = 1;

    while (cin >> n && n != 0) {
        for (int i = 0; i < n; i++) {
            cin >> arr[i];
        }

        // Si ya está ordenado desde el inicio, la salida es directamente 0
        if (estaOrdenado()) {
            cout << "There are 0 swap maps for input data set " << caso_num << "." <<
endl;
        } else {
            mapas_minimos = 0;
            min_pasos = 999999; // Inicializamos con un valor infinito práctico

            buscar(0);

            cout << "There are " << mapas_minimos << " swap maps for input data set "
<< caso_num << "." << endl;
        }
        caso_num++;
    }

    return 0;
}

```

3. Explicación del Código

La solución se diseñó bajo un principio de máxima simplicidad, evitando librerías complejas y utilizando estructuras de control básicas:

- **Estructura de Datos Estática:** Se emplea un arreglo global fijo `arr[5]` para almacenar los elementos de cada caso de prueba, lo cual emula el estilo directo de manipulación de arreglos del código base de referencia.
- **Función de Verificación:** La función `estaOrdenado()` recorre el arreglo de izquierda a derecha. Si detecta que cualquier elemento en la posición `i` es estrictamente mayor que el elemento en `i + 1`, interrumpe el ciclo y devuelve falso de inmediato.
- **Lógica del Backtracking:** La función recursiva `buscar(int pasos)` explora el espacio de estados de los intercambios. Para optimizar la ejecución, si los `pasos` actuales superan la variable de control `min_pasos`, la rama se corta inmediatamente (poda).
- **Gestión de Mínimos:** Cuando se detecta que el arreglo se ha ordenado con éxito, ocurren dos escenarios: si la cantidad de pasos actuales es estrictamente menor que `min_pasos`, se actualiza el récord mínimo y se reinicia el contador de mapas válidos a `1`; si es igual, simplemente se incrementa el contador `mapas_minimos` ya que se descubrió una ruta alternativa de idéntico tamaño mínimo.
- **Intercambio Manual y Backtrack:** Dentro del bucle recursivo, el intercambio se efectúa de manera manual empleando una variable auxiliar entera (`temp`). Inmediatamente después de la invocación recursiva, se vuelve a aplicar el mismo intercambio para restaurar el arreglo a su estado previo, garantizando la integridad de los datos para las siguientes iteraciones del bucle.